Data Wrangling and Visualisations

Set up

We always load the following packages.

```
library(fpp3)
library(tidyverse)
```

1. Downloading and wrangling data

Let's start by downloading a dataset on energy consumption from Kaggle. Save it into the raw data folder. Assuming that it is already saved, we can load it as follows.

```
elec_consumption <- read_csv("../_data/raw/AEP_hourly.csv")
```

```
Rows: 121273 Columns: 2
── Column specification
────────────────────────────────────────────────────────
Delimiter: ","
dbl  (1): AEP_MW
dttm (1): Datetime

ℹ Use `spec()` to retrieve the full column specification for
this data.
ℹ Specify the column types or set `show_col_types = FALSE` to
quiet this message.
```

Try turning this into a tsibble.

```
elec_consumption |> as_tsibble()
```

Something went wrong... why? Let's try to parse the error message:

```
Using `Datetime` as index variable.
Error in `validate_tsibble()`:
! A valid tsibble must have distinct rows identified by key and
ℹ Please use `duplicates()` to check the duplicated rows.
Run `rlang::last_trace()` to see where the error occurred.
```

Seems like there may be multiple entries with the same datetime. Let us

check:

```
duplicates(elec_consumption)
```

```
Using `Datetime` as index variable.

# A tibble: 8 × 2
  Datetime            AEP_MW
  <dttm>               <dbl>
1 2014-11-02 02:00:00  12994
2 2014-11-02 02:00:00  13190
3 2015-11-01 02:00:00  10785
4 2015-11-01 02:00:00  10542
5 2016-11-06 02:00:00  10964
6 2016-11-06 02:00:00  11008
7 2017-11-05 02:00:00  10596
8 2017-11-05 02:00:00  10446
```

The duplicate entries all occur in early November at 2am. This must be because of daylight savings.

How to deal with the duplicates? Probably the most “correct” way would be to “undo” daylight savings, but there is no built-in functionality for this, so we will just perform a stop-gap and take the mean of all measurements with the same datetime.

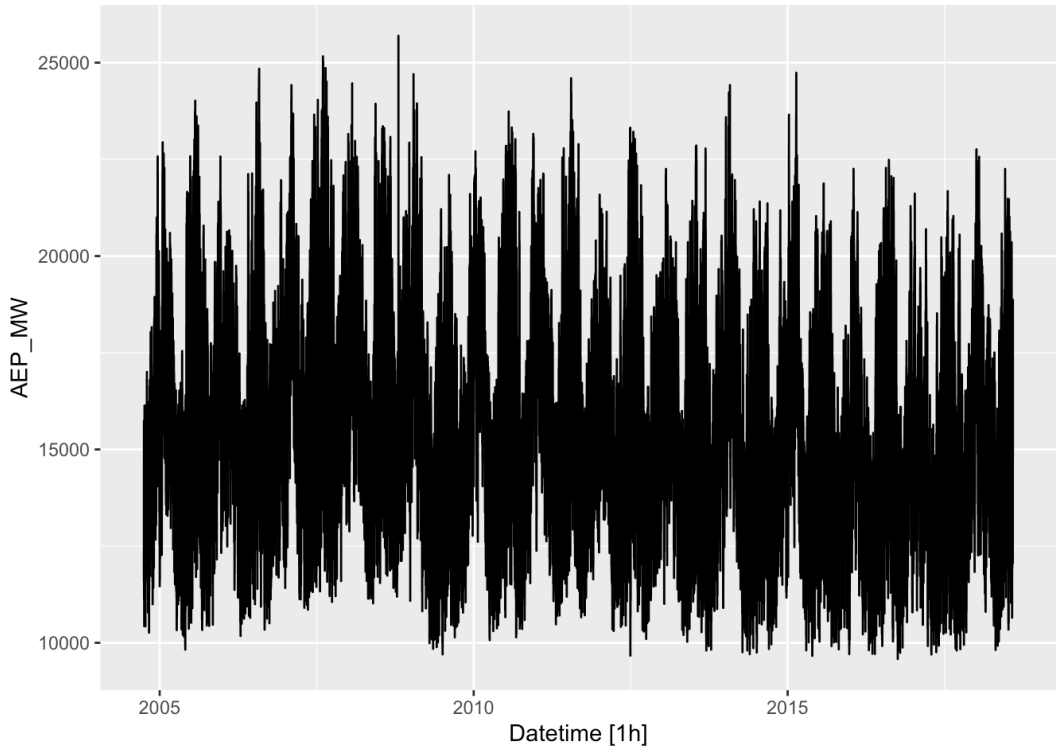
```
elec_consumption <- elec_consumption |>
  group_by(Datetime) |>
  summarise(AEP_MW = mean(AEP_MW)) |>
  as_tsibble(index = Datetime)
```

2. Time plots

Let us now compute a time plot:

```
elec_consumption |> autoplot()
```

```
Plot variable not specified, automatically selected `.vars =
AEP_MW`
```



There is clearly too much going on here. What can we do to visualize the time series better? 1. Filter the time series so that it is over a shorter time period. 2. Decompose the time series into seasonal, trend, and remainder components (next lesson) 3. Make a seasonal plot. 4. Aggregate the time series so that the duration between measurements is larger (e.g. days instead of hours).

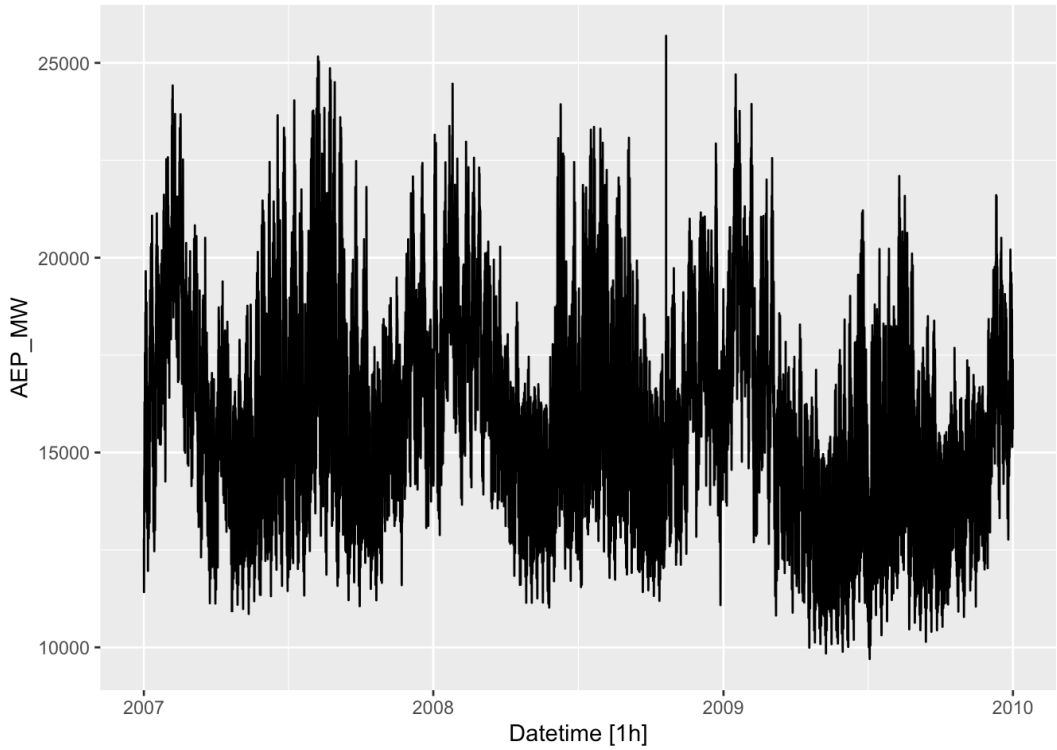
Let's try to perform a filter

```
elec_consumption |>
  filter(Datetime < 2010) |>
  autoplot()
```

What went wrong? Turns out inequality operations don't work naturally with datetime objects. We first need to extract the year number using the `year()` function.

```
elec_consumption |>
  filter(year(Datetime) < 2010 & year(Datetime) > 2006) |>
  autoplot()
```

```
Plot variable not specified, automatically selected `.vars =
AEP_MW`
```



There's actually a function from `tsibble` called `filter_index()` that provides more convenient ways of filtering based on the time index.

This plot is still quite noisy, so let's make seasonal plots.

3. Seasonal plots

```
elec_consumption |>
  gg_season()
```

Oops, there's an error again. Let's inspect the error message.

```
Plot variable not specified, automatically selected `y = AEP_MW`
! data contains implicit gaps in time. You should check your da
Backtrace:
 1. feasts::gg_season(elec_consumption)
 2. feasts:::check_gaps(data)
```

It says that there are gaps in the time index. This is unsurprising, because of daylight savings. To see where the gaps are, use `scan_gaps()`:

```
elec_consumption |>
  scan_gaps()
```

```
# A tsibble: 27 x 1 [1h] <UTC>
```

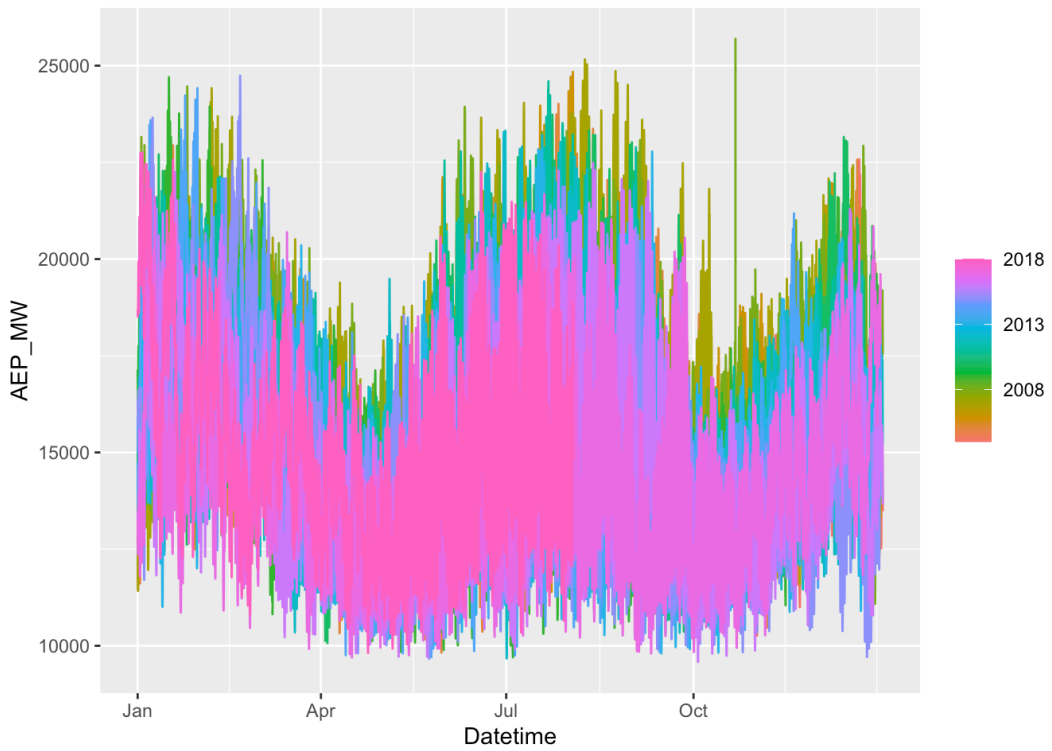
```
   Datetime
   <dttm>
 1 2004-10-31 02:00:00
 2 2005-04-03 03:00:00
 3 2005-10-30 02:00:00
 4 2006-04-02 03:00:00
 5 2006-10-29 02:00:00
 6 2007-03-11 03:00:00
 7 2007-11-04 02:00:00
 8 2008-03-09 03:00:00
 9 2008-11-02 02:00:00
10 2009-03-08 03:00:00
# ℹ 17 more rows
```

Now let us fill the gaps and make the seasonal plot.

```
elec_consumption |>
  fill_gaps() |>
  gg_season()
```

```
Warning: `gg_season()` was deprecated in feasts 0.4.2.
ℹ Please use `ggtime::gg_season()` instead.

Plot variable not specified, automatically selected `y =
AEP_MW`
```

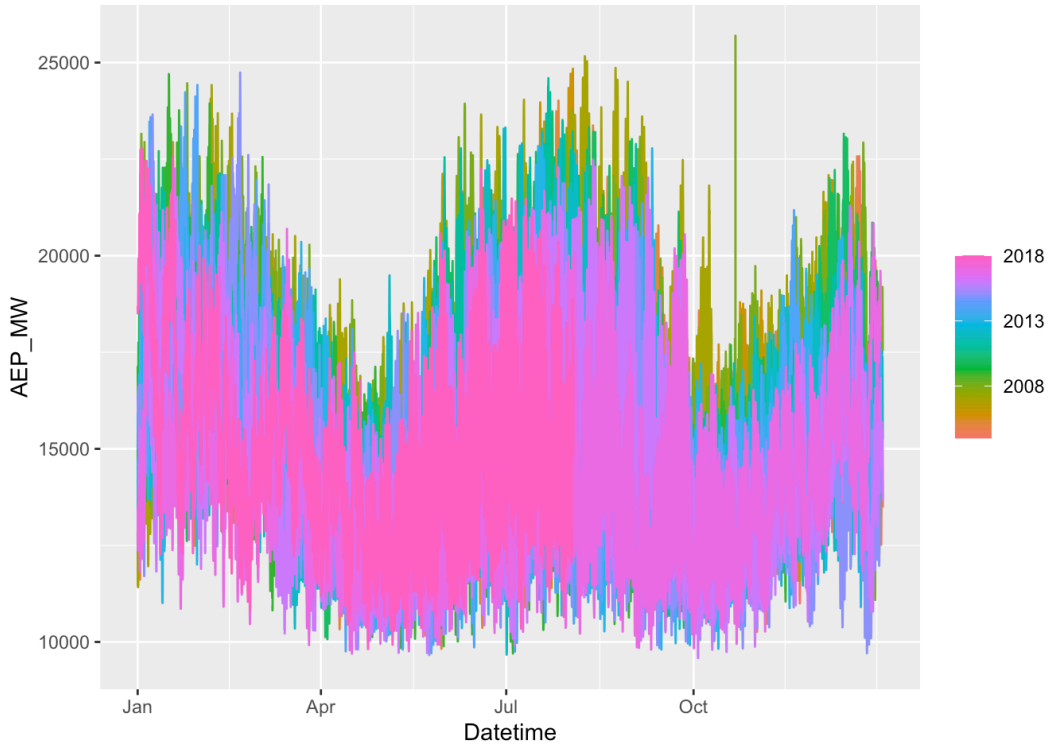


What period did `gg_season()` automatically select? It selected a period of

a year (i.e. 365 * 24).

```
elec_consumption |>
  fill_gaps() |>
  gg_season(period = "year")
```

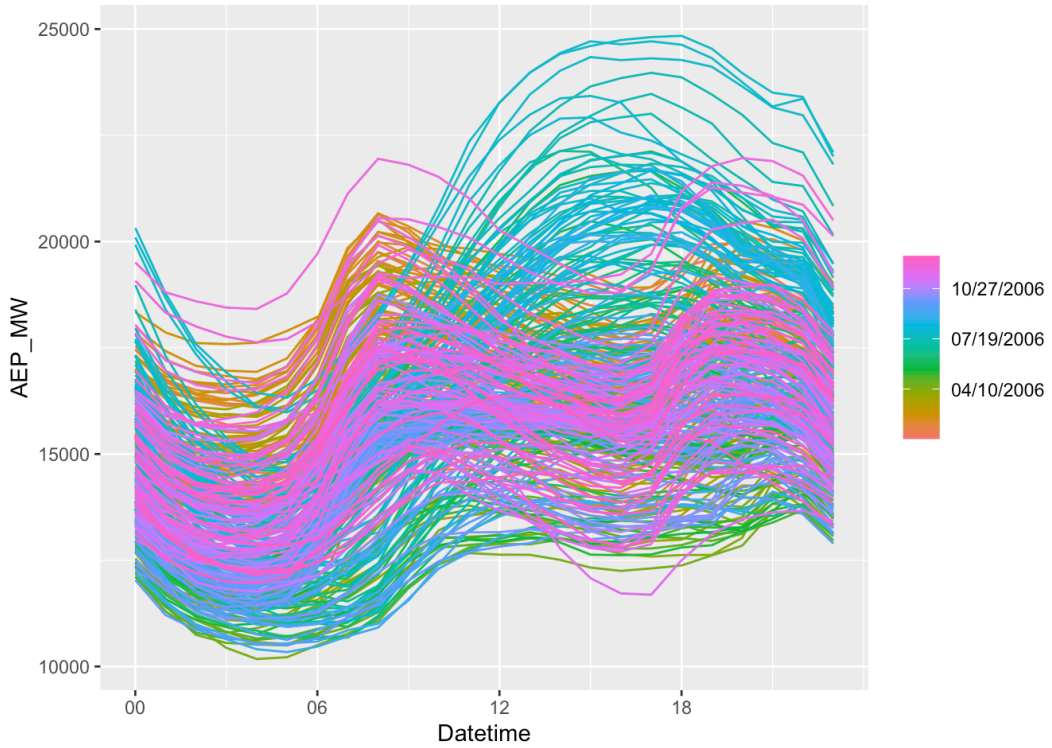
```
Plot variable not specified, automatically selected `y =
AEP_MW`
```



Manually setting the period to a year gives the same plot. We can try to interpret this seasonality. The highest energy consumption occurs in January and August, corresponding to the middle of Winter and Summer respectively. This is when people make the most of heaters or air-conditioning.

```
elec_consumption |>
  fill_gaps() |>
  filter(year(Datetime) == 2006) |>
  gg_season(period = "day")
```

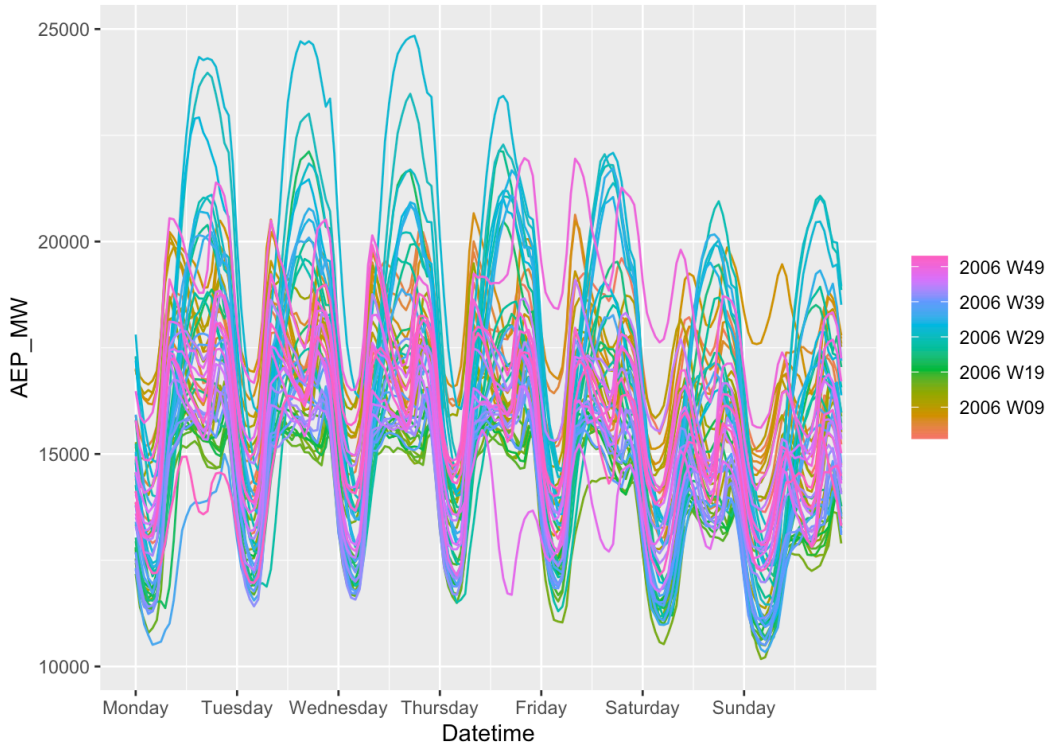
```
Plot variable not specified, automatically selected `y =
AEP_MW`
```



There seems to be daily seasonality. Energy consumption decreases after 9pm, bottoming out around 5am, when people are sleeping. It then increases again during the day. Note also the different patterns for Summer months (blue) and Winter months (pink/orange).

```
elec_consumption |>
  fill_gaps() |>
  filter(year(Datetime) == 2006) |>
  gg_season(period = "week")
```

```
Plot variable not specified, automatically selected `y =
AEP_MW`
```



There seems to be weekly seasonality too. Energy consumption is lower on the weekends than on weekdays. Note that the fluctuations over each day is part of the daily seasonality, not the weekly seasonality.

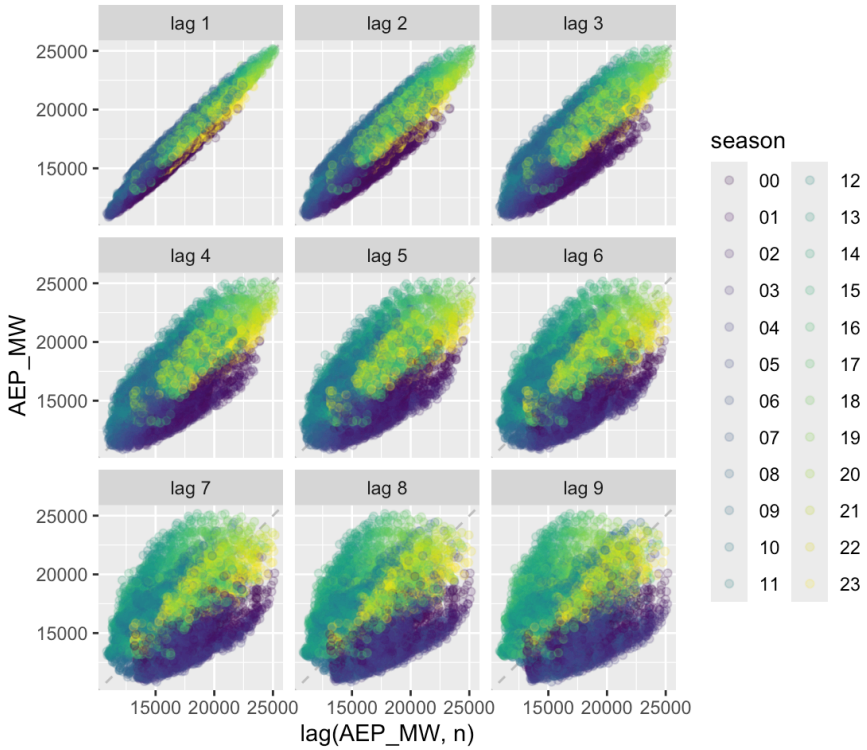
4. Lag plots

Make a lag plot of `elec_consumption`. What issues arise? How can you make the plot look like what was shown in the lecture video?

```
elec_consumption |>
  filter(year(Datetime) == 2007) |>
  gg_lag(geom = "point", alpha = 0.2)
```

```
Warning: `gg_lag()` was deprecated in feasts 0.4.2.
ℹ Please use `ggtime::gg_lag()` instead.

Plot variable not specified, automatically selected `y =
AEP_MW`
```



We also experience some overplotting. We can reduce overplotting by filtering and adding transparency (alpha).